- *The user is unauthenticated*—The user is redirected to the login page.
- *The user is authenticated but doesn't satisfy the policy*—The user is redirected to a "forbidden" or "access denied" page.

These three outcomes correlate with real-life outcomes you might expect when trying to pass through security at the airport:

- *You have a valid boarding pass*—You can enter security as normal.
- *You don't have a boarding pass*—You're redirected to purchase a ticket.
- *Your boarding pass is invalid (you turned up a day late, for example)*—You're blocked from entering.

Listing 24.3 shows how you can apply a policy to a Razor Page using the `[Authorize]` attribute, but you still need to define the `CanEnterSecurity` policy.

You add policies to an ASP.NET Core application in Program.cs, as shown in listing 24.4. First, you add the authorization services and return an `AuthorizationBuilder` object using `AddAuthorizationBuilder()`. You can then add policies to the builder by calling `AddPolicy()`. You define the policy itself by calling methods in a lambda method on a `AuthorizationPolicyBuilder` (called `policyBuilder` here).

Listing 24.4 Adding an authorization policy using `AuthorizationPolicyBuilder`